ITF23019: Parallel and Distributed Programming

Spring Semester, 2026

Lab5: When to Use Parallel Programming

Submission Deadline: February 14th, 2026 23:59

You need at least 50 points to pass this lab assignment.

When we have a **big** problem which can take a significant amount of time to run, we can think of dividing it into a number of sub-problems and having them executed simultaneously (by different cores) or concurrently (by a single core) to reduce execution time. While doing so, we expect that the parallel (or concurrent) program can efficiently solve the problem within in a reasonable time i.e, runs faster than its serial counterpart. However, how can we say that the problem being solved is big and when can we write a parallel program instead of a serial program?

When we say that a problem is "big" or "small," we are referring to two main factors:

- Problem size, meaning the amount of input data associated with the task.
- Time complexity, meaning the number of computations required to solve the problem.

The two factors are the nature of the problem being solved.

It is worth noting that in a parallel program, communication among tasks, threads or processes can be a significant portion of the overhead. However, handling communication may require a synchronization mechanism to avoid data inconsistency which is beyond the scope of this lab. This lab only focuses on the impacts of data size and time complexity of the input problem to execution time.

1 Problem size

Problem size refers to the scale or magnitude of the input data. For example, in sorting algorithms, the problem size could be the number of elements in the array to be sorted. In graph algorithms, it might be the number of vertices and edges in the graph. If you have to calculate the summation of an array with `N` elements, then the problem size is defined by the value of `N`. If you calcualte the multiplication of two square matrices with size `N` $\times$ N, the problem size is also defined as N.

2 Time complexity of the input problem

In theoretical computer science, the time complexity is the computational complexity that describes the amount of computer time it takes to run an algorithm. Time complexity is commonly estimated by counting the number of elementary operations performed by the algorithm, supposing that each elementary operation takes a fixed amount of time to perform. Thus, the amount of time taken and the number of elementary operations performed by the algorithm are taken to be related by a constant factor[1].

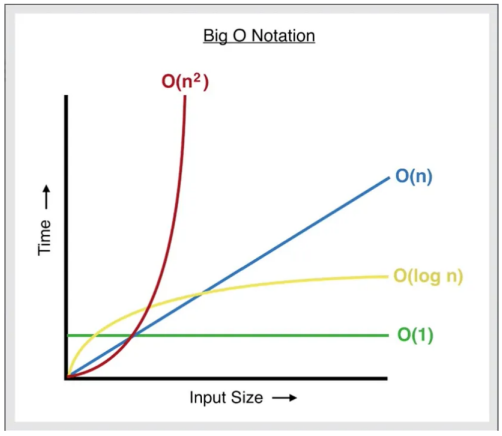

Figure 1: Running time vs input size. Note that $O(N^3) > O(N^2)$..

Since an algorithm's running time may vary among different inputs of the same size, one commonly considers the *worst-case time complexity*, which is the maximum amount of time required for inputs of a given size. Less common, and usually specified explicitly, is the *average-case complexity*, which is the average of the time taken on inputs of a given size (this makes sense because there are only a finite number of possible inputs of a given size). In both cases, the time complexity is generally expressed as a function of the size of the input. Since this function is generally difficult to compute exactly, and the running time for small inputs is usually not consequential, one commonly focuses on the behavior of the complexity when the input size increases—that is the asymptotic behavior of the complexity. Time complexity is often expressed using Big O notation, $\mathcal{O}()$[1] Let's consider the following pseudo-code to calculate the summation of N elements in an array:

```
int sum = 0;
for(int i = 0; i < N; i++) {
   sum += array[i];
}
```

The number of computations is N, therefore, the time complexity of the algorithm is $O(N)$, meaning that the running time of the algorithm grows **linearly** with the size of the input.

Common time complexities include (refer to Figure 1) :

- $\mathcal{O}(1)$ - Constant time complexity: the running time remains constant regardless of the input size.
- $\mathcal{O}(logn)$ - Logarithmic time complexity: the running time grows logarithmically with the input size.
- $\mathcal{O}(nlogn)$- Linearithmic time complexity: common for efficient sorting algorithms like Merge Sort and Heap Sort.
- $\mathcal{O}(n^2)$- Quadratic time complexity: the running time grows quadratically with the input size. Quick sort and bubble sort has time complexity of $\mathcal{O}(n^2)$.

It is worth noting that a low complexity does not necessarily mean the problem is simple and vice versa. In practices, we can have a problem which is simple to implement but time complexity is high. For instance, bubble sort is simple to understand and easy to implement but its complexity is $\mathcal{O}(n^2)$ while merge sort is more difficult to implement but has lower time complexity, $\mathcal{O}(nlogn)$. Another example is to calculating

the nth number in Fibonacci series. If we use recursive programming, the code is very simple with few lines of code; however, the number of computations is very high.

```
int f(int n) {
   if (n == 0) return 0;
   if (n == 1) return 1;
      return f(n - 1) + f(n - 2);
}
```

3 Whether or not the problem can be parallelized

To decide if we should write a parallel program or a serial program to solve a problem, we also have to see whether or not the problem can be parallelized. A problem can be parallelized only when it can be broken into independent sub-problem that can run simultaneously. If each step depends on the result of the previous step, the problem is inherently serial, and running it on multiple processors won't make it faster. For example, computing each element of a vector dot product is independent, so the work can be split across many threads. But tasks like parsing a file, updating a game loop, or running Dijkstra's algorithm require information from earlier steps, so they must be executed in order. Another example is the Fibonacci series above. The calculation of the Fibonacci sequences as shown would entail dependent calculations rather than independents ones. The calculation of the nth number uses those of both the (n-1)th and (n-2)th numbers. These three terms cannot be calculated independently and therefore, not in parallel.

Parallelism is powerful, but it's not magic. Even in problems that can be parallelized, only the independent tasks speeds up — the serial part still limits performance. This is why identifying tasks dependencies is the first step in deciding whether parallel programming is worth the effort. If most of the work can be done independently, parallelism can give huge gains. If not, a well-written serial program may actually run faster and be much simpler to maintain. In addition, parallelism adds overhead to the program which the serial program does not have.

In summary, using a parallel program is beneficial when the problem can be divided into independent tasks and involves heavy computation, large input data, or both. When a problem is inherently sequential, simple, or small in scale, a serial implementation is usually the better choice, as parallelism would add unnecessary overhead without improving performance.

4 Design a parallel program

How a developer designs a parallel program has strong impacts to its execution time. In practices, this involves partitions of the problem being solved and communication among divided tasks. In this lab assignment, partitions refers to data partitions and work partition for different tasks/threads. (It also relates to communication among tasks but we do not focus on task interaction in this lab). The key point for partition is to make the work load among different threads/processes as the same as possible (load balancing). There may be several ways for partitioning when we parallelized a serial implementation. Figure 3 shows two possible partitions of a `Matrix - Vector Multiplication` problem, each results in different running time.

In Partition 1, each task computes one element of the output matrix, i.e., Task 1 computes y[0]. Task 1 owns the first row of matrix A and shares vector b with all other tasks. This is the same for all tasks. In Partition 2, each task computes three elements of the results vector, i.e., Task 1 computes y[0], y[1], and y[2]. Task 1 owns the first three rows of matrix A and shares vector b with all other tasks. In this particular example, the workload for each task in both partitions is the same. However, the running time of the two partitions can be different.

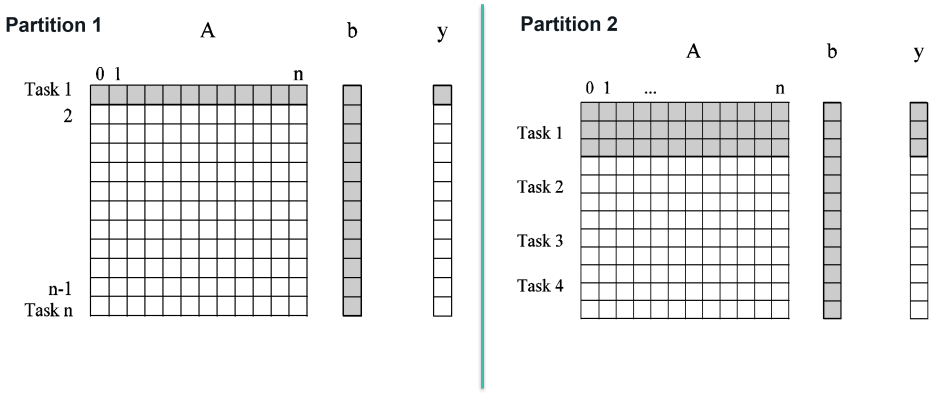

Figure 2: Two possible partitions.

4.1 Matrix - Matrix Multiplication

This subsection is used for Exercise 2. Matrix - Matrix multiplication is one of the most basic operations that you can do with matrices. In mathematics, particularly in linear algebra, it is an operation that produces a matrix from two input matrices[1]. If you have a matrix A with size $\mathbf{M} \times \mathbf{N}$ (i.e, $\mathbf{M}$ rows and $\mathbf{N}$ columns) and another matrix B with size $\mathbf{N} \times \mathbf{P}$ (i.e, $\mathbf{N}$ rows and $\mathbf{P}$ columns), you can multiply both matrices and obtain a matrix C with size $\mathbf{M} \times \mathbf{P}$ (i.e, $\mathbf{M}$ rows and $\mathbf{P}$ columns). The product of matrices A and B is denoted as AB.

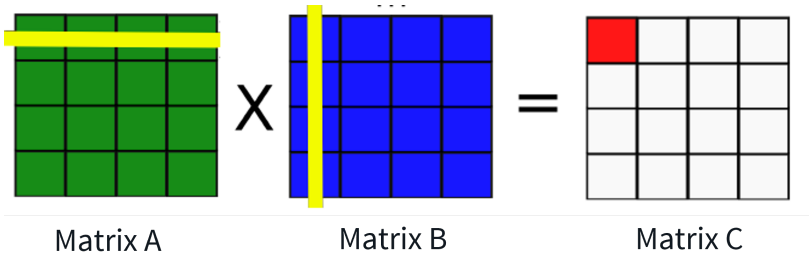

Figure 3: Matrix - matrix multiplication

$$\begin{bmatrix} C_{00} & C_{12} & \dots & C_{1P} \\ C_{10} & C_{22} & \dots & C_{2P} \\ \dots & \dots & \dots & \dots \\ C_{(M-1)0} & C_{M2} & \dots & C_{MP} \end{bmatrix} = C. \tag{1}$$

$$AB = \begin{bmatrix} A_{00} & A_{01} & \dots & A_{0(N-1)} \\ A_{10} & A_{11} & \dots & A_{1(N-1)} \\ \dots & \dots & \dots & \dots \\ A_{M0} & A_{M1} & \dots & A_{(M-1)(N-1)} \end{bmatrix} \begin{bmatrix} B_{00} & B_{01} & \dots & B_{1(P-1)} \\ B_{11} & B_{11} & \dots & B_{2(P-1)} \\ \dots & \dots & \dots & \dots \\ B_{(N-1)0} & B_{(N-1)1} & \dots & B_{(N-1)(P-1)} \end{bmatrix} \tag{2}$$

$$= \begin{bmatrix} C_{00} & C_{01} & \dots & C_{0(P-1)} \\ C_{10} & C_{12} & \dots & C_{1(P-1)} \\ \dots & \dots & \dots & \dots \\ C_{(M-1)0} & C_{(M-1)1} & \dots & C_{(M-1)(P-1)} \end{bmatrix} = C. \tag{3}$$

where

$$C_{ij} = A_{i0}B_{0j} + A_{i1}B_{1j} + \cdots + A_{i(N-1)}B_{(N-1)j} = \sum_{k=0}^{(N-1)} A_{ik}B_{kj} \quad (4)$$

The algorithm complexity is $O(N^3)$ which is very high - as shown in Figure 1. When both complexity and size of input problem are high, we expect that the parallel version is much better than its serial version. The pseudo-code for calculating matric C is

```
for (i = 0; i < M; i ++)
    for (j = 0; j < N; j ++)
      for (k = 0; k < P; k ++)
         C_{ij} + = A_{ik}A_{kj}
```

5 Exercises

5.1 Exercise 1 (50 points)

In this exercise, you will implement a task that compute dot product (multiplication) of two input vectors vectors. Each vector is generated once you run the program with the below code snippet:

```
public static int[] arrayGen(int N){
   int[] a = new int[N];
   for (int i = 0; i <N; i++){
      a[i] = ThreadLocalRandom.current().nextInt(10);
   }
   return a;
}
```

The size of the array (i.e, the number of elements in the array) can be varied by changing input argument `N` in `arrayGen()`. The incompleted code is provided in `vector_vector_mul` project. This project consists of three packages with four classes :

- parallel package consists of `VectorVectorMulTask` class and `VectorVectorMulParallel` class. The `VectorVectorMulTask` implements a parallel task in the parallel program. The `VectorVectorMulParallel` creates a ForkJoin pool and invokes the multiplication task.
- serial package consists `VectorVectorMulSerial` class which implements a task in the serial program.
- `vector_vector_mul` package consists of the `Main` class which generates two vectors, runs the serial/parallel programs and produces other expected outputs.

Your tasks for this exercise are:

- Use `ForkJoin` framework to complete parallel implementation in `VectorVectorMulTask.java` and `VectorVectorMulParallel.java`
- When the output of your parallel program implementation and that of the serial program are the same, write code the `Main` class to compute the speedup of your parallel program. Include the output in your report.
- Fix the value of **threshold** in the VectorVectorMulTask class and change the size of the two input vectors to see how the speedup varies. Include several outputs in your report.

- Run your implementation with different values of **threshold** while keeping the size of the input vectors fixed to see how the speedup varies. Include several outputs in your report.
- Set an extreme value for the vector size and observe what happens when you run your program.

5.2 Exercise 2 (50 points)

In this exercise, you will write a Java program to compute multiplication of two square matrices. These two input matrics are generated when the program runs.

```
private static int[][] generate2DArray(int N, int M){
   System.out.println("generate2DArray ....");
   Random random = new Random(20260203);

   int[][] a = new int[N][M];
      for (int i = 0; i < N; i++)
         for(int j = 0; j < M; j++)
            a[i][j] = random.nextInt(10);
   return a;
}
```

In the above code, when the two input arguments are the same (`N = M`), you will have a square matrics.

The starter code of the serial and parallel implementations is in `matrix-matrix-mul` project. This project consists of three packages with four classes:

- `matrix_matrix_mul` package consists of the `Main` class which runs the serial/parallel programs and produces other expected outputs.
- `serial` package consists of `MatrixMatrixMulSerial` class which implements the serial program.
- `parallel` package consists `MatrixMatrixMulTask` and `MatrixMatrixMulParallel` classes. The `MatrixMatrixMulTask` implements a parallel task. The `MatrixMatrixMulParallel` generates two matrices, creates a ForkJoin pool and invokes the multiplication task.

There are several ways to partition tasks. However, a reasonable partition is as follows: each task computes a certain number of elements in the output matrix and owns a certain number of rows in the first matrix. All tasks share the second matrix. For example if task 1 is responsible for the first row of the result matrix, it will own the first row of the first input matrix and share the second matrix with all other tasks. In this way, tasks do not need to communicate with each other.

Your tasks for this exercise are:

- Complete serial implementation.

Hint: pseudo-code for the elements of the output matrix at row `i` and column `j` (results[i][j]) is:

```
for(int i = 0; i < N; i++)
   for(int j = 0; j < P; j++)
      for(int k = 0; k < M; k++)
         result[i][j] += matrix1[i][k]*matrix2[k][j];
```

- Use `ForkJoin` framework to complete parallel implementation. (The provided code uses `RecursiveAction` to implement `Task` in the ForkJoin framework but you are free to use `RecursiveTask` - modify the starter code accordingly.)

- The `compareResults(resultSerial, resultParallel)` in the starter code is used to check whether your serial and parallel implementations produce the same output. If they do, your two implementations are correct, and you can proceed to write your code to compute the speedup of your parallel program. Include the output in your report. Otherwise, you need to debug your code to fix your implementation until both serial and parallel versions are correct.
- Change the size of input matrics (i.e, `N, M, P` in the starter code) and the value of **threshold** in `MatrixMatrixMulTask` class to see how speedup varies. Include the output of your experiments in the report and explain in which cases your parallel version runs faster than your serial version.
- The run complexity of **Exercise 1** is $\mathcal{O}(n)$ while that of **Exercise 2** is $\mathcal{O}(n^3)$. Suppose that your serial and parallel implementations in both two excercises are resonable, and that you use the same data size (for example, N elements in each vector in Exercise 1 and N rows/columns in each matrix in Exercise 2). Which exercise results in a better speedup? Explain your answer.

Note:

- Make sure that your code runs before you commit your codes in your lab repository.
- If you use any generative AI tool for the exercises in this lab or future labs, you need to acknowledge that in your lab report.
- You can also collaborate with friends. If it is the case, you also need to acknowledge your collaboration in your report.

6 References

1. Time complexity Wikipedia

7 What To Submit

Complete the exercises in this assignment. Then, put your codes along with lab5_report file (Markdown or pdf file) inside the lab5 directory of your repository. Commit your changes and remember to check the GitHub website to make sure all files have been submitted.